3D Data Science with Python

```
for point in point_cloud:
    # 12 million times. see you tomorrow ☕
```

Sergio Peral · ML Engineer, Teqyard
github.com/Sloppy1999/pycon-nl-2026-3d-data-science



3D used to mean C++. Not anymore

C++ with PCL

```
using Cloud =
    pcl::PointCloud<pcl::PointXYZ>;
Cloud::Ptr cloud(new Cloud);
Cloud down;
if (pcl::io::loadPCDFile(
    "crop.pcd", *cloud)) return 1;
pcl::VoxelGrid<pcl::PointXYZ> vg;
vg.setInputCloud(cloud);
vg.setLeafSize(0.5f, 0.5f, 0.5f);
vg.filter(down);
std::cout << cloud->size()
  << " -> " << down.size() << "\n";


# output
722961 -> 230519
```

Python with Open3D

```
pcd = o3d.io.read_point_cloud(path)
down = pcd.voxel_down_sample(0.5)
print(len(pcd.points), "->",
      len(down.points))


# output
722961 -> 232180


# install
uv add open3d "laspy[lazrs]" pyvista
```

Python is the interface. C++ does the work



Every call you make here runs compiled code

- NumPy: a compiled C core, with some C++
- Open3D: C++, wrapped for Python
- PyVista: VTK, which is C++
- laspy: the lazrs decoder is Rust
- Python gives you the interface; the heavy loops run below it

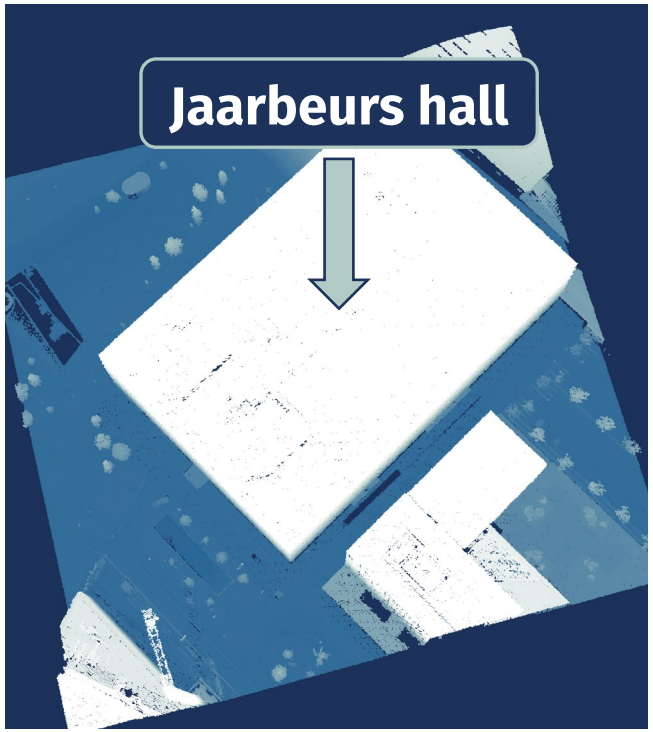
Meme template: the Scooby-Doo unmasking scene (Hanna-Barbera). Python and C++ logos belong to their owners; no endorsement implied.

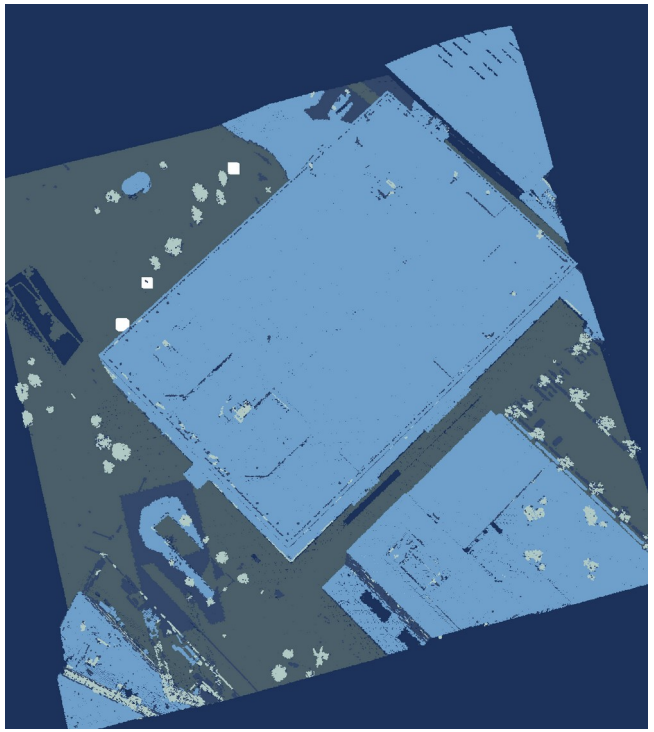
The Python 3D ecosystem at a glance



Blue outline: used in today's pipeline. Logos belong to their owners; no endorsement implied. 119 libraries checked on PyPI and GitHub: docs/ECOSYSTEM.md in the repo.

Today's case: the Jaarbeurs hall, from a plane





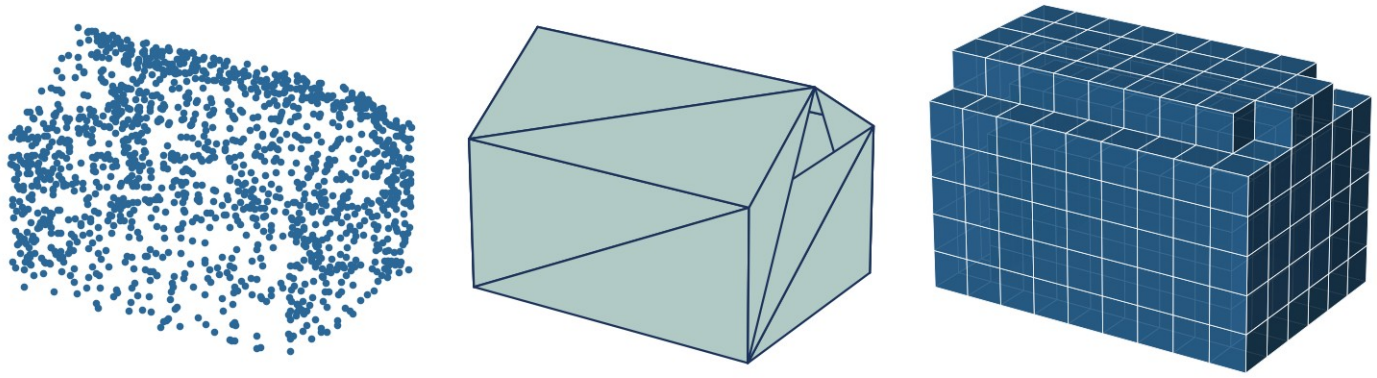
The data set

Area	200 m × 200 m
Points	722 961
Density	about 18 per m²
Source	AHN5, ahn.nl
Flown	April 2023, night
File	LAZ, 4.3 MB
Coordinates	RD New + NAP
Licence	CC BY 4.0

Goal: ground, buildings, vegetation, street furniture; scored against AHN.

Point cloud: AHN5, Actueel Hoogtebestand Nederland (ahn.nl), Het Waterschapshuis, CC BY 4.0. AHN file 135000_455000, cropped and processed.

One object, three representations



Point cloud

1500 × 3 floats
no connectivity

Mesh

10 vertices, 16 triangles
a surface with neighbours

Voxel grid

182 occupied 0.5 m cells
volume at one resolution

Synthetic house, drawn three ways: 1 500 sampled points, 10 vertices and 16 triangles, 182 voxels of 0.5 m.

Points measure, meshes connect, voxels index space. Convert only when the task needs it.

- Point cloud: an unordered N×3 array plus attributes; no neighbours stored.
- Mesh: vertices plus faces, so surfaces, normals and areas are defined.
- Voxel grid: occupied cells at one resolution; detail below a cell is gone.

A 3D project moves through six stages



Acquisition
Scan the scene
laspy

1

2

Preparation
Clean and register
NumPy, Open3D

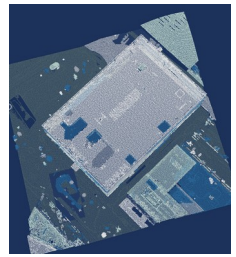
Application
Ship it

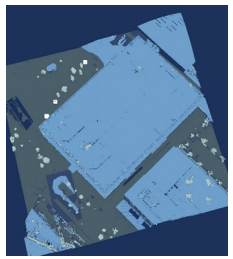
6

Loop back when the data surprises

3

Exploration
Model and label
Open3D, SciPy





Visualisation
See it, trust it
PyVista

5

4

Analysis
Measure, monitor
NumPy, SciPy

Point cloud: AHN5, Actueel Hoogtebestand Nederland (ahn.nl), Het Waterschapshuis, CC BY 4.0. AHN file 135000_455000, cropped and processed.

Four rules of thumb for 3D data

1 Think in arrays, not points

A point cloud is an N×3 table plus attribute columns.

2 Look before you compute

Render it; check units, coordinates and classes.

3 Make it small first

Crop, tile and downsample; scale up last.

4 Let compiled code loop

One call per array or tile, never per point.

No single library does the whole job

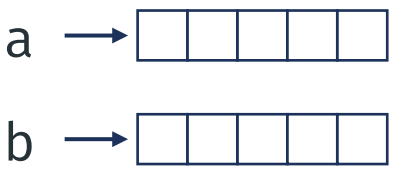
Load	Voxel downsampling	Ground removal	Clustering	Labelling	Visualisation
laspy LAS/LAZ in and out	**Open3D** downsampling, planes, DBSCAN				**PyVista** off-screen renders
			SciPy KDTree batch neighbour queries		
NumPy the arrays every step passes on					

Not in this pipeline

trimesh
meshes, hulls, oriented boxes

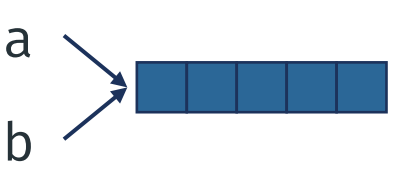
PyTorch3D
differentiable 3D for training

Python passes the array, not the points



copy

new memory: costs time and RAM



view

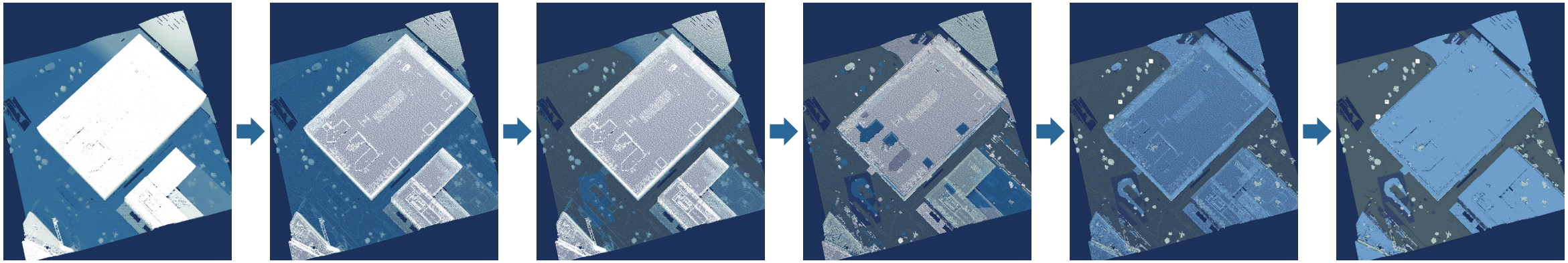
same memory: free, writes show through

`las.xyz`	**copy**	new array, every access
`Vector3dVector(xyz)`	**copy**	float32: 39× slower
`np.asarray(pcd.points)`	**view**	writes reach the point cloud
`pv.PolyData(pts)`	**view**	same memory as pts

Each xyz copy costs 24 B per point: 0.5 GB per 1 km AHN file.

Conditions: Ryzen 7 5700X, Python 3.13.15, NumPy 2.5.3; 722 961 points, median of 7 runs; AHN5 crop (CC BY 4.0). Full: results/python_facts.json.

Six steps turn a point cloud into labels



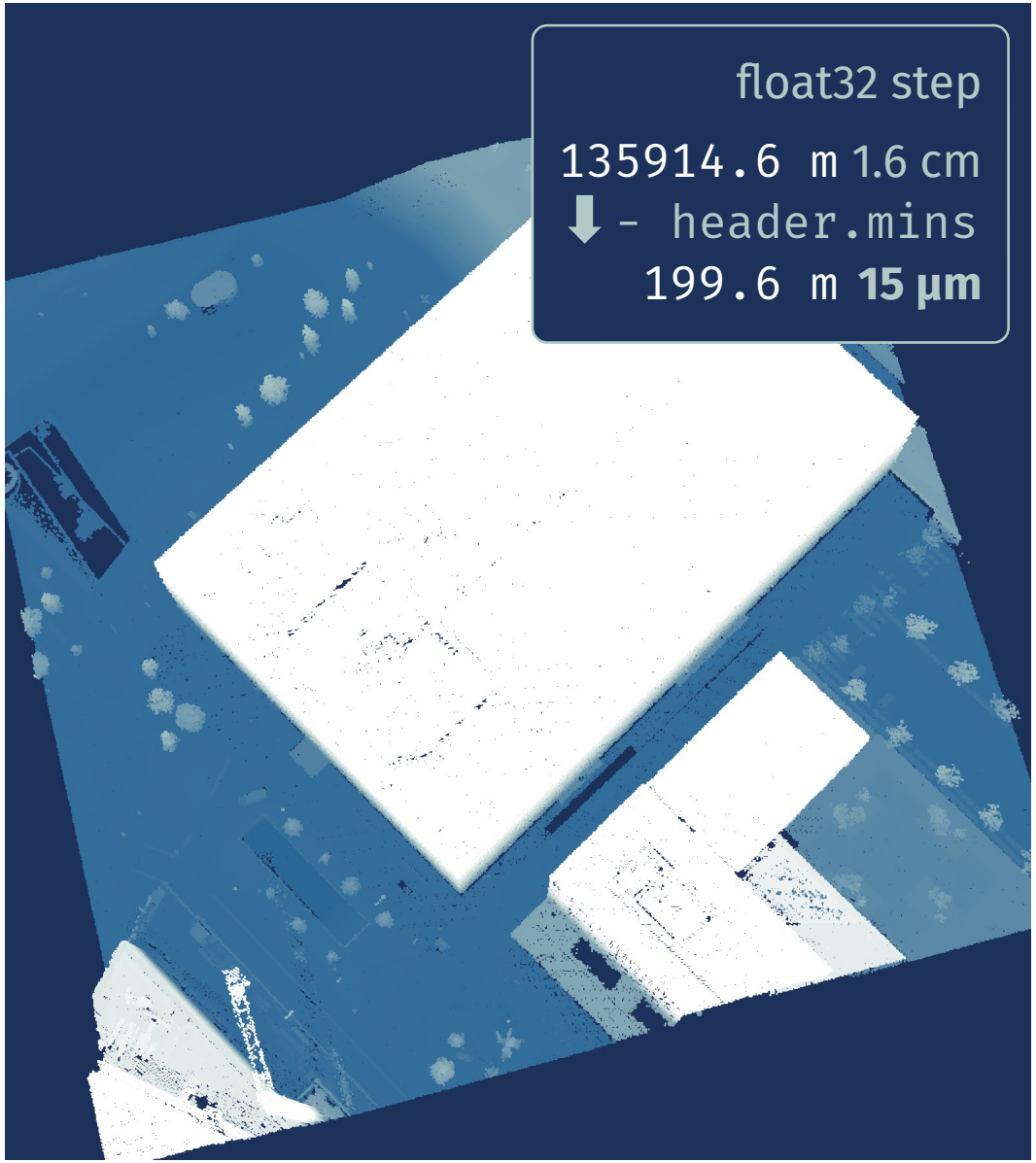
Load	**Voxel downsampling**	**Ground removal**	**Clustering**	**Labelling**	**Visualisation**
laspy	Open3D	Open3D	Open3D, SciPy	NumPy, SciPy	SciPy, PyVista

Point cloud: AHN5, Actueel Hoogtebestand Nederland (ahn.nl), Het Waterschapshuis, CC BY 4.0. AHN file 135000_455000, cropped and processed.

1 Load:
laspy hands you NumPy

```python
import laspy
import open3d as o3d
las = laspy.read("crop.laz")
xyz = las.xyz - las.header.mins
v = o3d.utility.Vector3dVector(xyz)
pcd = o3d.geometry.PointCloud(v)
```

Subtract a local origin before anything else.



Point cloud: AHN5, Actueel Hoogtebestand Nederland (ahn.nl), Het Waterschapshuis, CC BY 4.0. AHN file 135000_455000, cropped and processed.

2 Voxel downsampling: one average per voxel

```
down = pcd.voxel_down_sample(0.5)
P = np.asarray(down.points)
print(len(xyz), "->", len(P))
```

722 961 points in,
232 180 out at 0.5 m voxels.

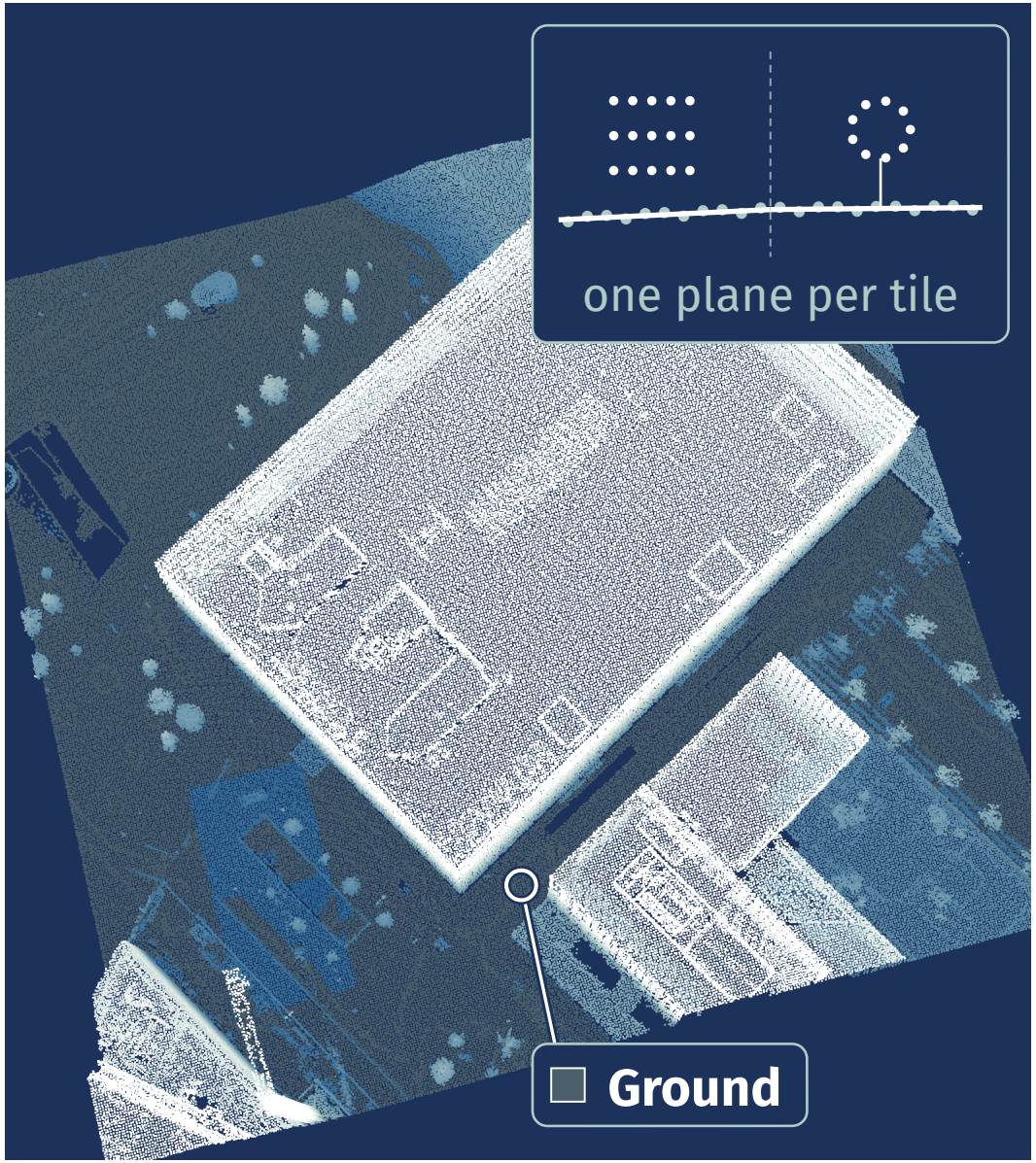


Point cloud: AHN5, Actueel Hoogtebestand Nederland (ahn.nl), Het Waterschapshuis, CC BY 4.0. AHN file 135000_455000, cropped and processed.

3 Ground removal: one plane per tile

```
o3d.utility.random.seed(0)
ground = np.zeros(len(P), bool)
for i in tiles(P, 25):
    z0 = np.percentile(P[i, 2], 2)
    low = i[P[i, 2] < z0 + 1.5]
    t = down.select_by_index(low)
    _, inl = t.segment_plane(
        0.4, 3, 300,
        probability=1.0)
    ground[low[inl]] = z0 < 5
```

Scored against AHN's ground:
98 % precision, 94 % recall.

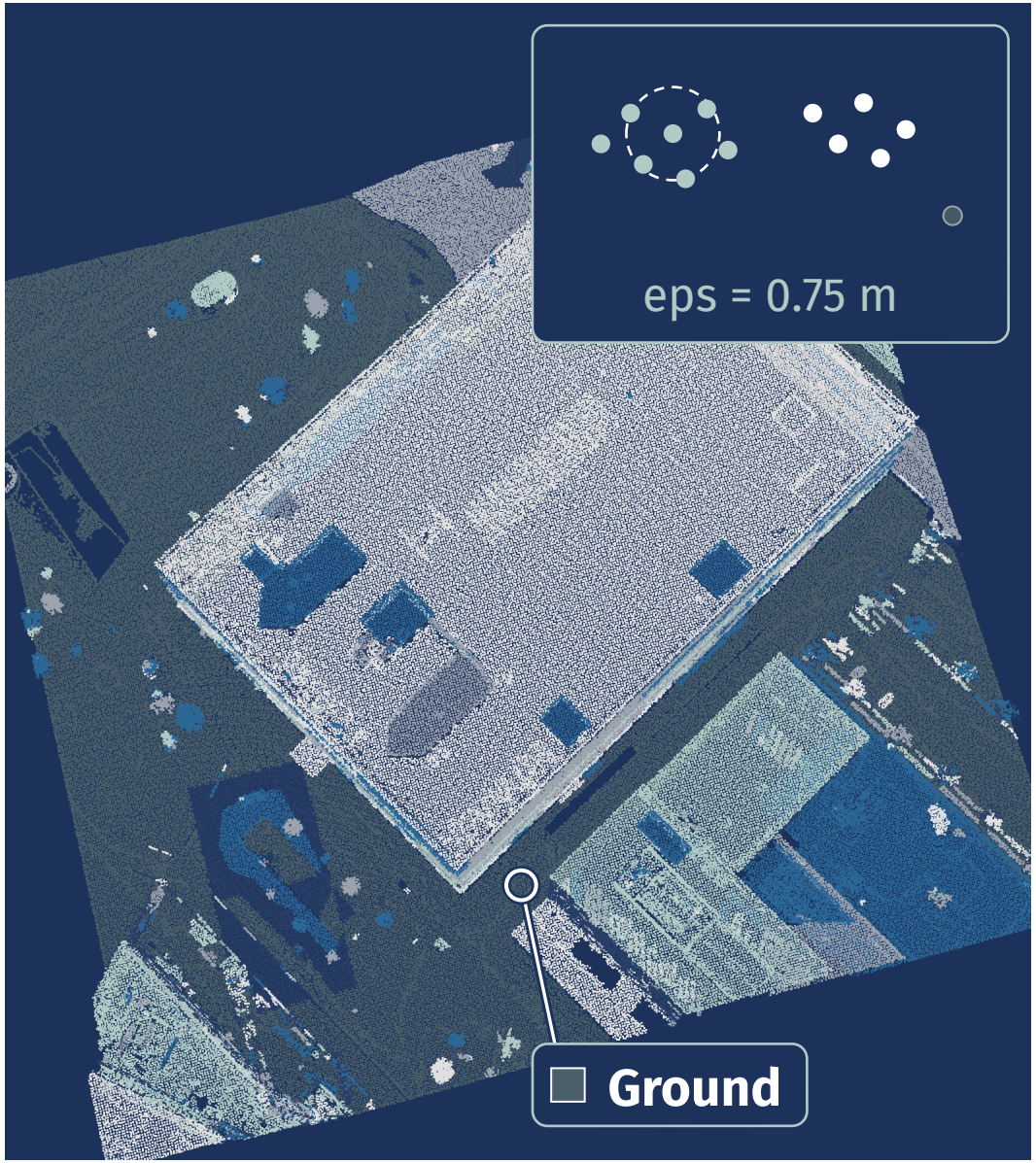


Point cloud: AHN5, Actueel Hoogtebestand Nederland (ahn.nl), Het Waterschapshuis, CC BY 4.0. AHN file 135000_455000, cropped and processed.

4 Clustering: DBSCAN groups what touches

```
hag = height_above(P, ground)
up = ~ground & (hag > 0.3)
rest = down.select_by_index(
    np.flatnonzero(up))
lab = rest.cluster_dbscan(
    eps=0.75, min_points=6)
ids = np.asarray(lab)
print(ids.max() + 1, "clusters")
```

1 024 clusters. Clusters are not labels yet.

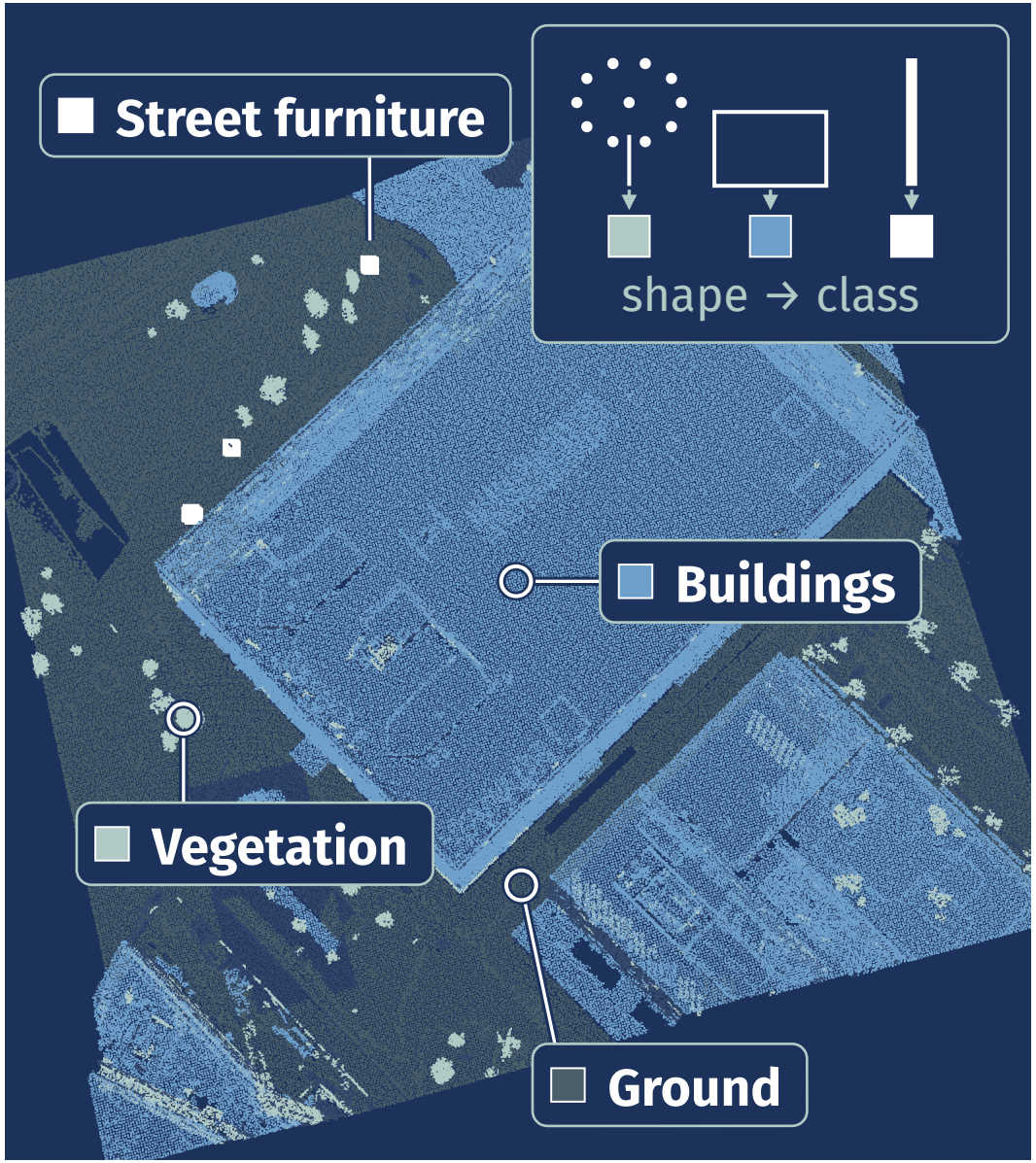


Point cloud: AHN5, Actueel Hoogtebestand Nederland (ahn.nl), Het Waterschapshuis, CC BY 4.0. AHN file 135000_455000, cropped and processed.

5 Labelling: rules name the clusters

```
def name(c):
    if c.multi > 0.4 and c.top > 1:
        return VEGETATION
    if c.foot > 25 and c.top > 2.5:
        return BUILDINGS
    if c.to_building < 2:  # facade
        return BUILDINGS
    if c.slender and c.free:
        return STREET_FURNITURE
    return UNLABELLED
```

Leaf-off scan: bare trees echo several times.



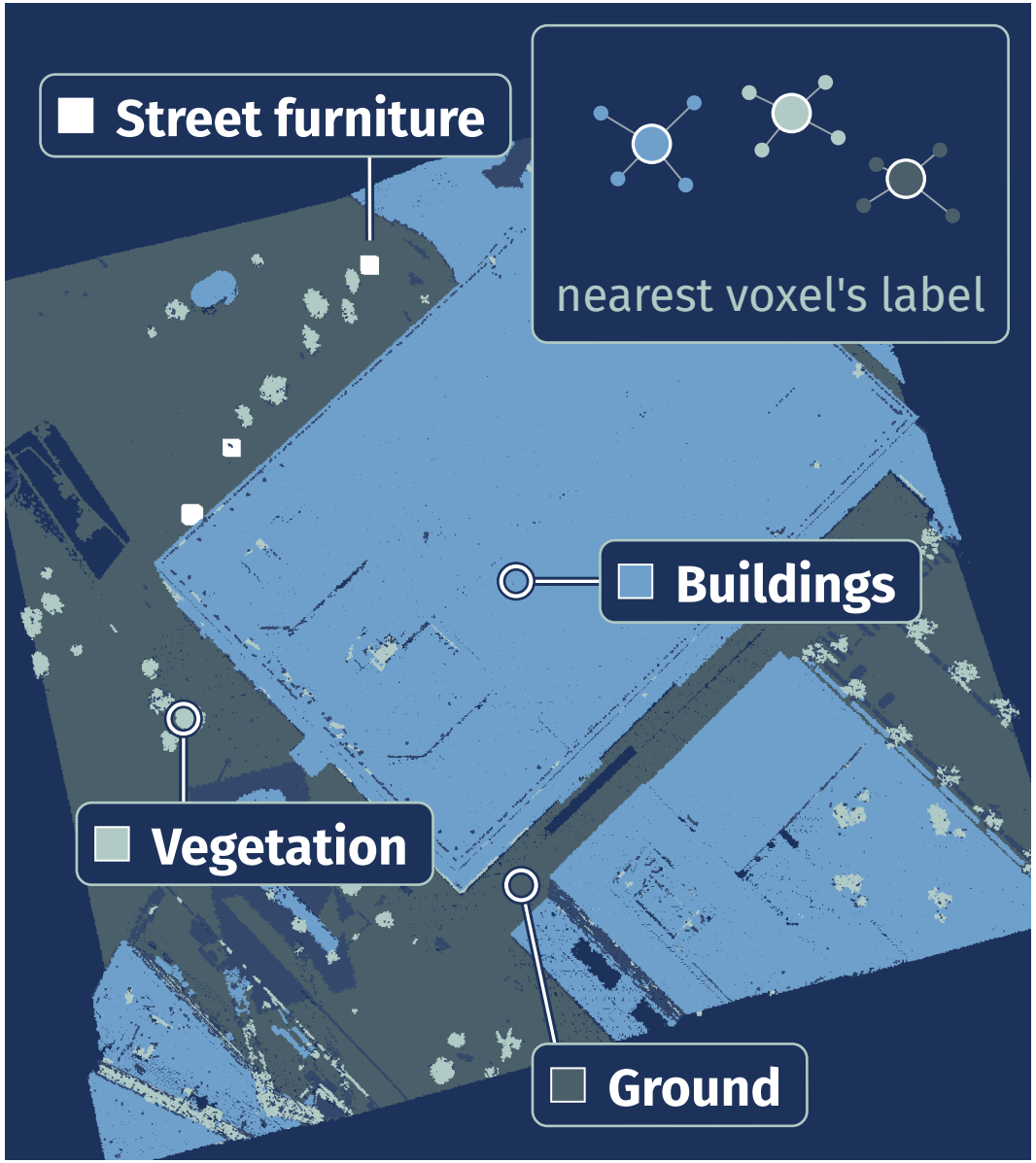
Street furniture: 3 objects, 53 points, drawn enlarged. Poles nearly vanish.

Point cloud: AHN5, Actueel Hoogtebestand Nederland (ahn.nl), Het Waterschapshuis, CC BY 4.0. AHN file 135000_455000, cropped and processed.

6 Visualisation: PyVista paints every point

```
P = np.asarray(down.points)
_, j = KDTree(P).query(
    xyz, workers=-1)
pl = pv.Plotter(off_screen=True)
pl.add_points(xyz, point_size=3,
    scalars=RGB[cls[j]], rgb=True)
pl.camera_position = CAM
pl.screenshot("f6_final.png")
```

Fixed camera, off-screen, one PNG per step.



Street furniture: 3 objects, 53 points, drawn enlarged. Poles nearly vanish.

Point cloud: AHN5, Actueel Hoogtebestand Nederland (ahn.nl), Het Waterschapshuis, CC BY 4.0. AHN file 135000_455000, cropped and processed.

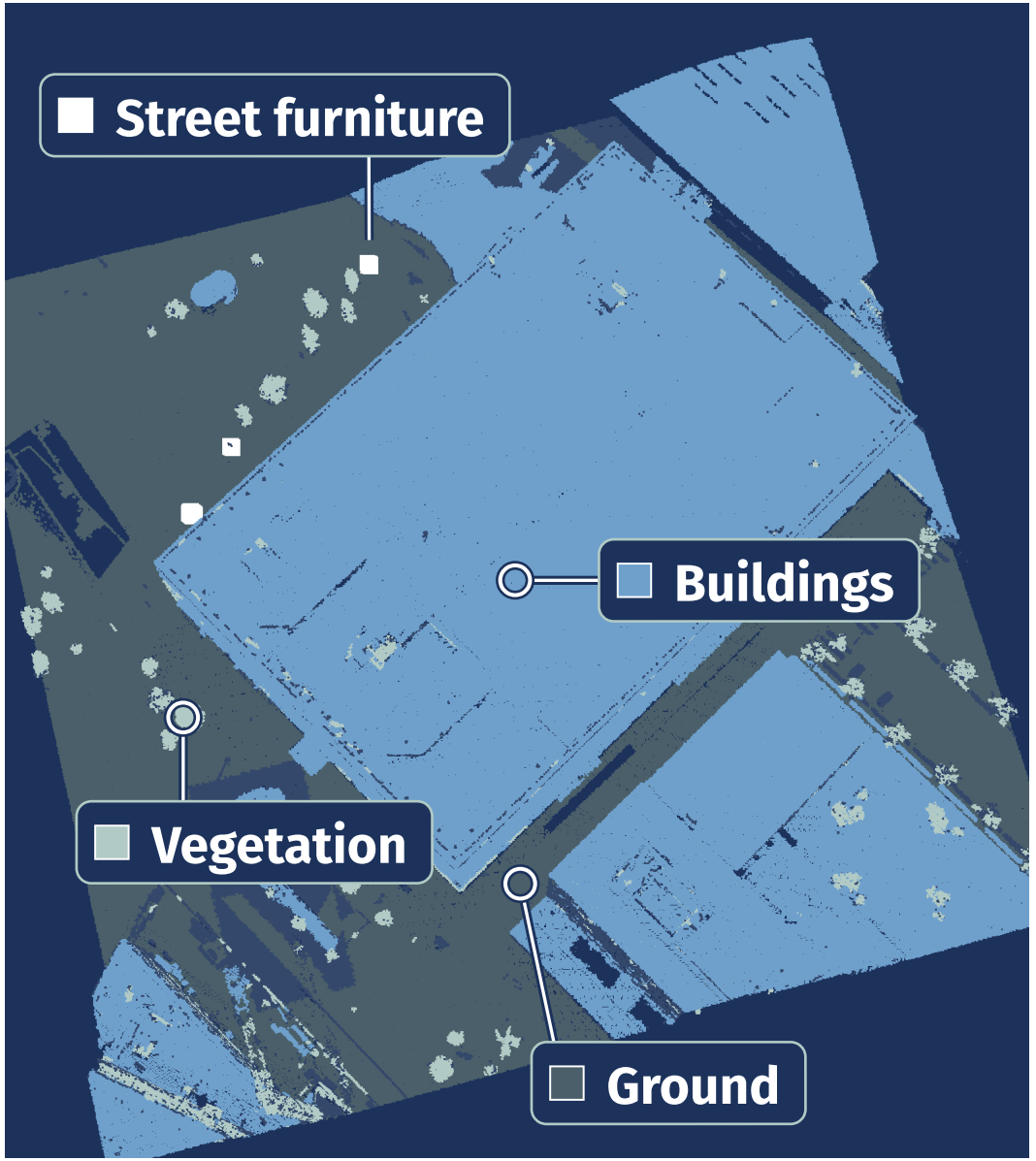
What the six steps did

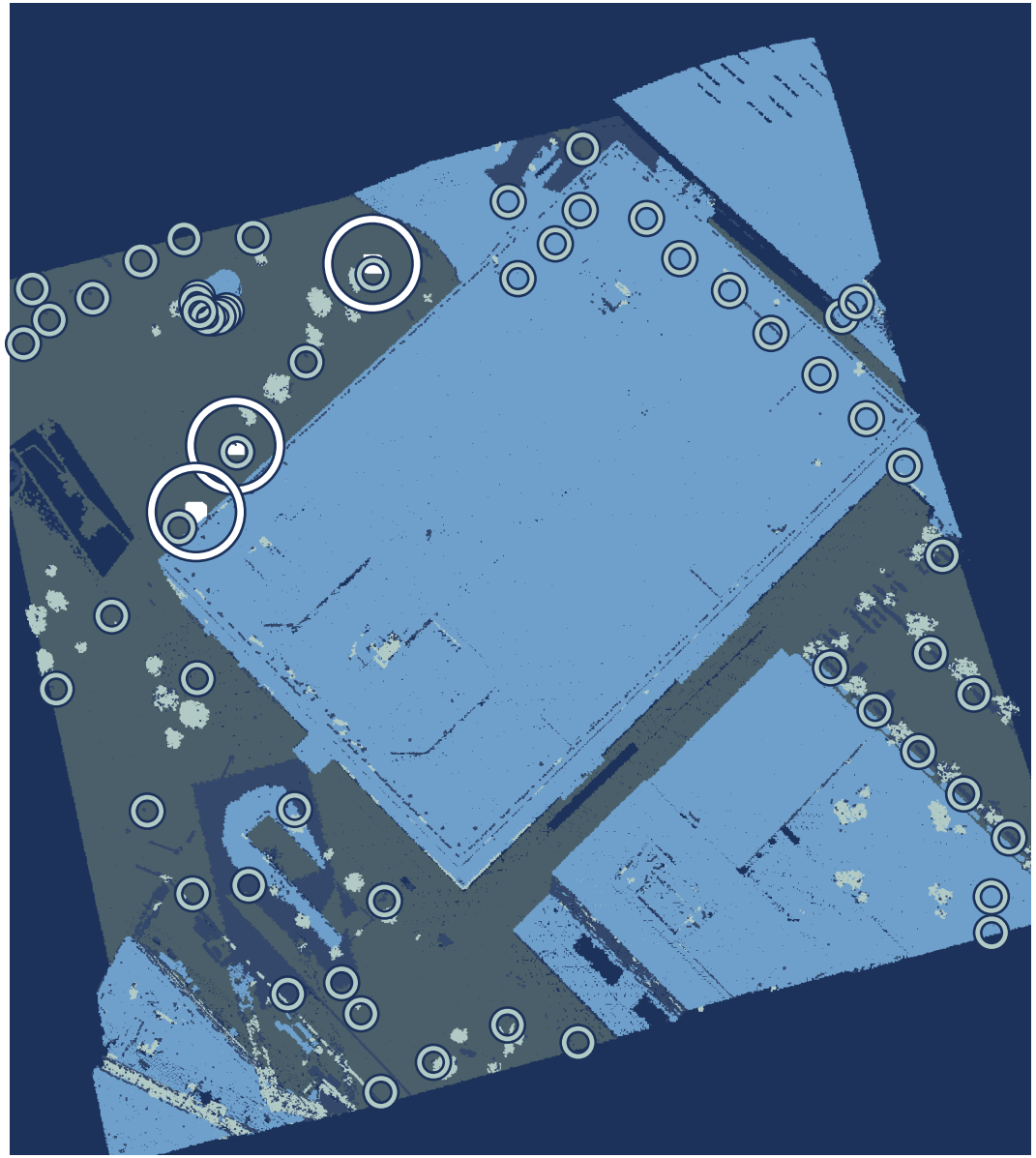
Step	Library	Result	Time
Load	laspy	722 961 points	57 ms
Voxels	Open3D	232 180 voxels	84 ms
Ground	Open3D	98 % precision	81 ms
Clusters	Open3D	1 024 clusters	137 ms
Labels	NumPy	4 classes	151 ms
Paint-back	SciPy	all points	99 ms

0.6 s from file to labels, and no loop over points.

One run, indicative; renders not included. Street furniture drawn enlarged.

Point cloud: AHN5, Actueel Hoogtebestand Nederland (ahn.nl), Het Waterschapshuis, CC BY 4.0. AHN file 135000_455000, cropped and processed.





Expectation vs reality: street furniture

Expectation: the BGT register

61 poles 51 lamp posts, 10 bollards

Reality: our pipeline

3 objects 2 of them on a lamp post



2021 · Robot8A · CC BY-SA 4.0



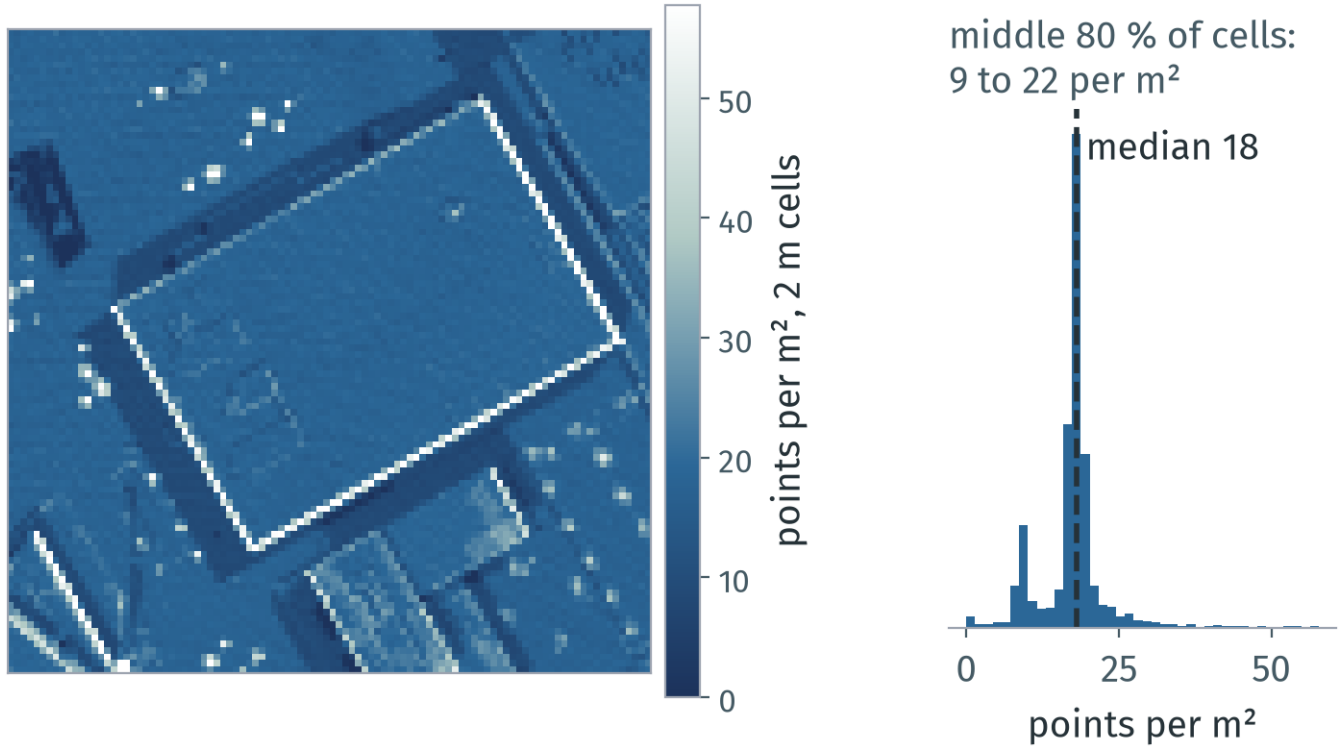
2014 · V. van Werkhooven · CC BY 3.0

42 of 61 poles get no LiDAR point above 1.5 m.

Poles: BGT (Kadaster, CC BY 4.0) via PDOK, 2026-10-12. Points: within 0.6 m, over 1.5 m above ground. Photos: Jaarbeursplein, Wikimedia Commons.

Point cloud: AHN5, Actueel Hoogtebestand Nederland (ahn.nl), Het Waterschapshuis, CC BY 4.0. AHN file 135000_455000, cropped and processed.

A point cloud samples the surface unevenly



Points per m² in 2 m cells, this crop (AHN5). Walls stack many points in one map cell.

Density, noise and gaps vary within one file. Measure them before tuning anything.

- Here the middle 80 % of cells hold 9 to 22 points per m²; median 18.
- Occlusion, flight-strip overlap, angle and material decide where points land.
- Every point carries error: AHN4 specifies at most 5 cm in height; AHN5 publishes no figure yet.

Quiz: which is faster, and by how much?

Distance of every point to the ground plane, 722 961 points.

A

```
a, b, c, d = plane
h = np.empty(len(P))  # distances
for i, p in enumerate(P):
    x, y, z = p[0], p[1], p[2]
    h[i] = abs(a*x + b*y + c*z + d)
```

B

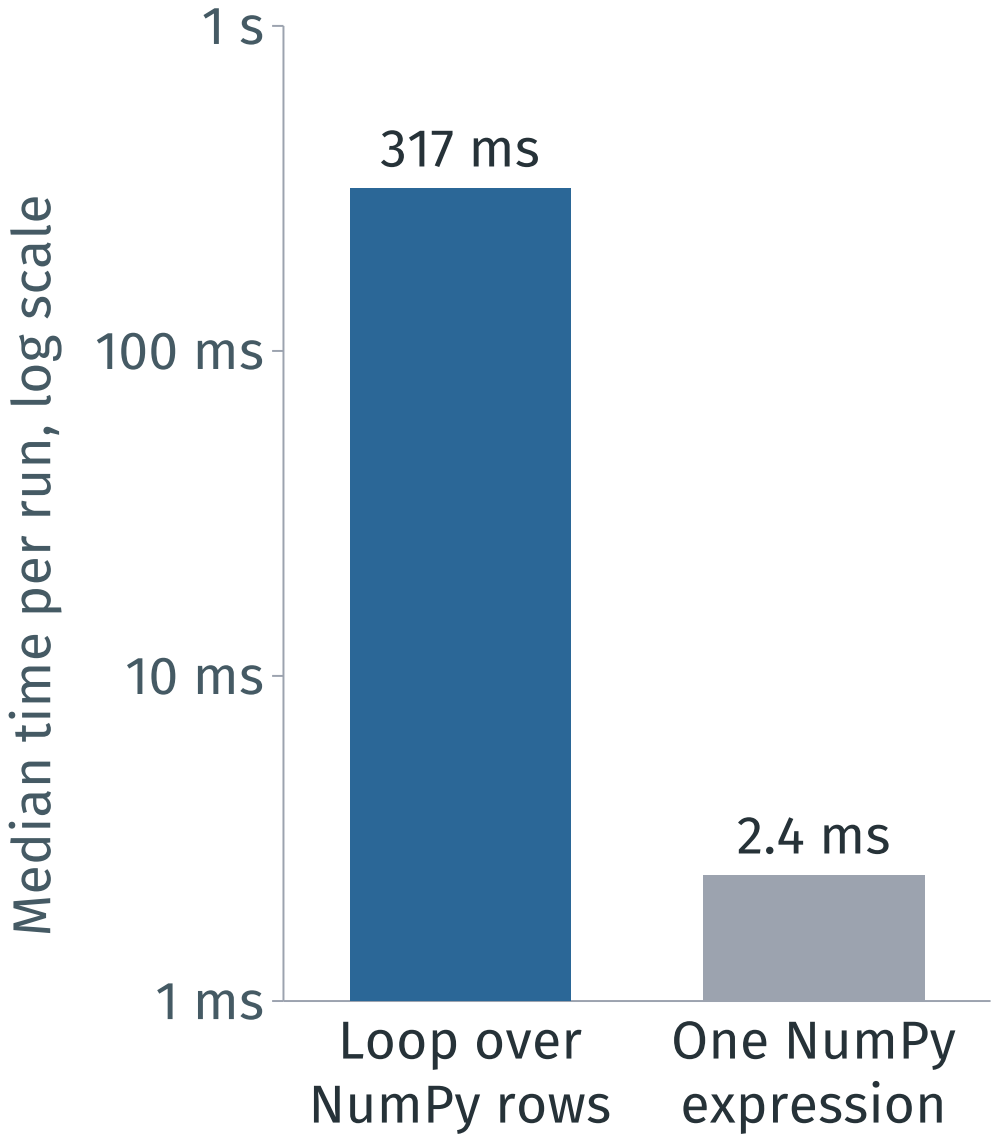
```
a, b, c, d = plane
x, y, z = P.T  # columns, no copy
h = np.abs(a*x + b*y + c*z + d)
```

The faster one wins by about: 2× 10× 100× 1000×

Both give identical results (np.array_equal). P: the crop's xyz from slide 12, column-ordered. plane: one seeded segment_plane on the whole crop.

How much slower is the loop over rows?

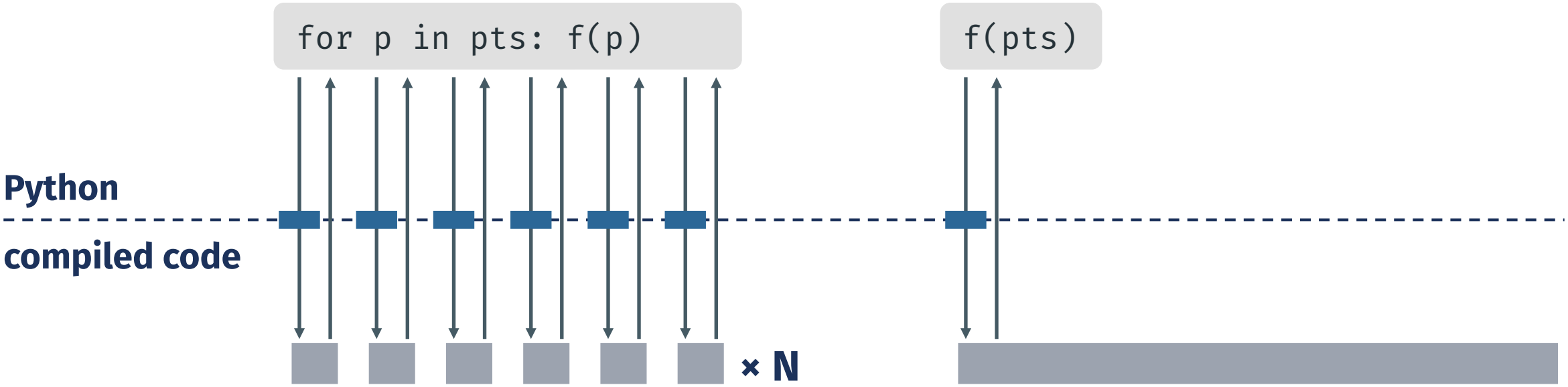
130× slower



Same data, same answer, far fewer calls into compiled code.

Conditions: Ryzen 7 5700X, Python 3.13.15, NumPy 2.5.3; 722 961 points, median of 7 runs; AHN5 crop (CC BY 4.0). Full: results/bench_cliff.json.

Every trip into compiled code pays a toll



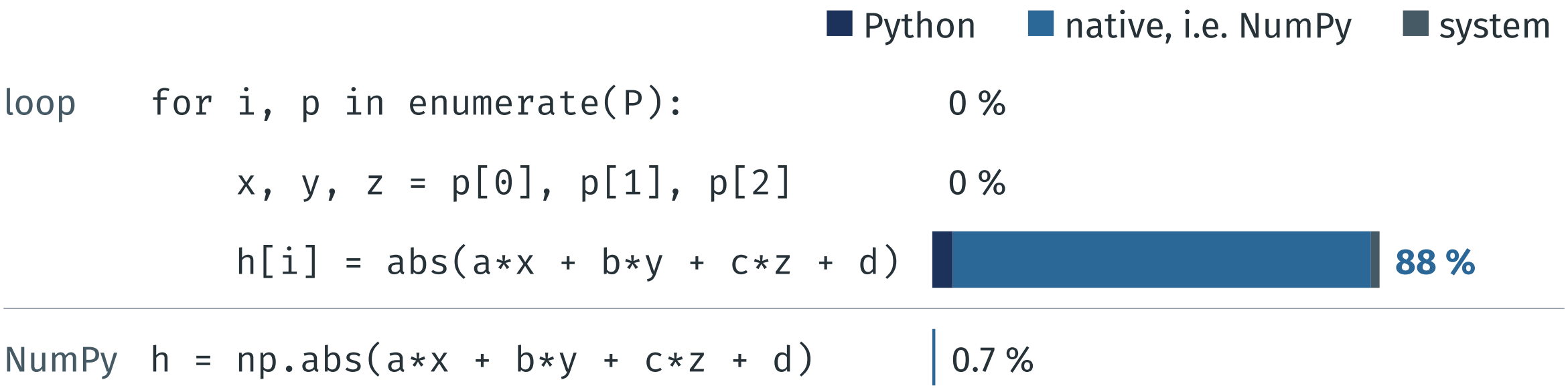
439 ns per loop step (several NumPy calls)

C++ libraries too: even an empty k = 16 KDTree query costs 9.8 µs.

Conditions: Ryzen 7 5700X, Python 3.13.15, NumPy 2.5.3; 722 961 points, median of 7 runs; AHN5 crop (CC BY 4.0). Full: results/bench_cliff.json.

Profile before you vectorise

```
$ scalene run --cpu-only loop.py
```

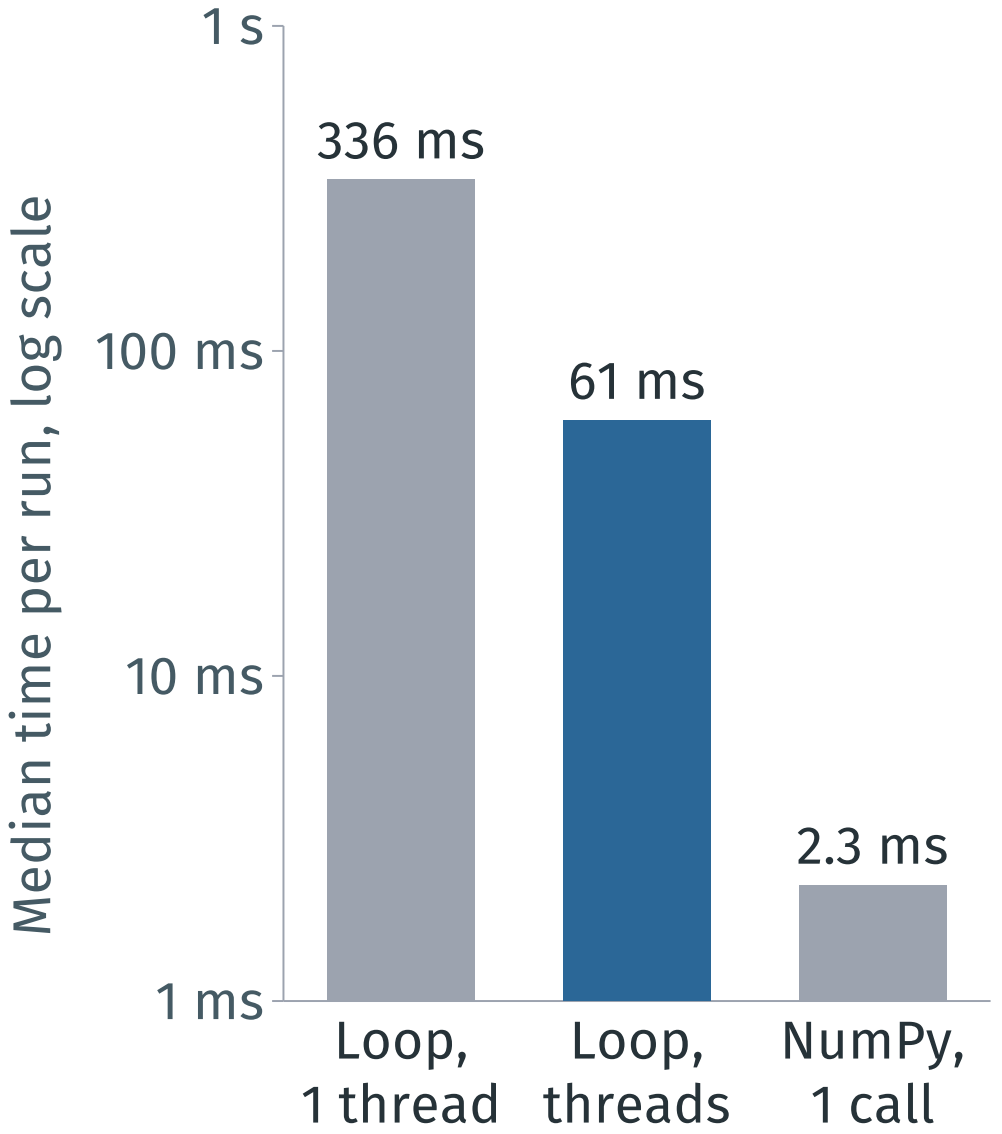


The loop's last line takes 88 % of the run. As one NumPy expression: 0.7 %.

Conditions: Ryzen 7 5700X, Python 3.13.15, Scalene 2.3.0 (--cpu-only, a sample every 1 ms); 722 961 points, both versions 30 times per run, median of 3 runs; AHN5 crop (CC BY 4.0). Shares are of the whole run, imports and reading the file included. Full: results/profile_scalene.json.

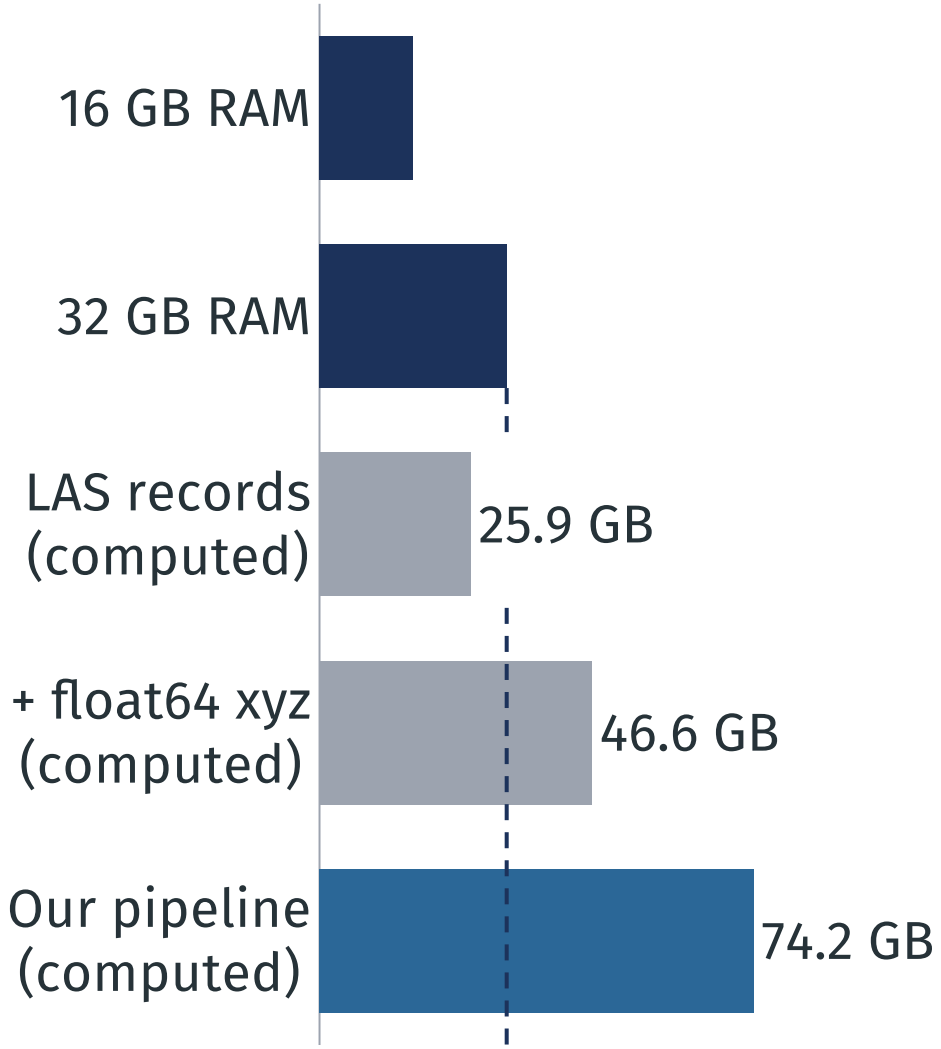
Will free-threaded Python save my loop?

5.5× faster on 8 free threads



With the GIL, threads gain nothing; one NumPy call is still 27× faster.

Conditions: Ryzen 7 5700X, CPython 3.14.7 free-threaded and 3.14.7, NumPy 2.5.4; 722 961 points, median of 7 runs; AHN5 crop (CC BY 4.0). Full: results/bench_freethreaded.json.



Loading this area's AHN file outgrows 32 GB

86 B per point in our pipeline

926 203 338 points × 86 B each.

Bytes per point measured on the 1 km AHN file 135000_455000 (21 013 926 points); totals for 2023_C_31HZ2.LAZ computed, not run. Our pipeline: load and voxel downsampling (slides 12 and 13), all arrays kept, after gc.collect() and malloc_trim(0); peak 154 B per point.

It works on my machine (31.3 GiB RAM).

Conditions: Ryzen 7 5700X, Python 3.13.15, NumPy 2.5.3; 21 013 926 points, median of 3 runs, each in a fresh process; AHN5 1 km file (CC BY 4.0). 1 GB = 2^{30} B. Full: results/memory.json.

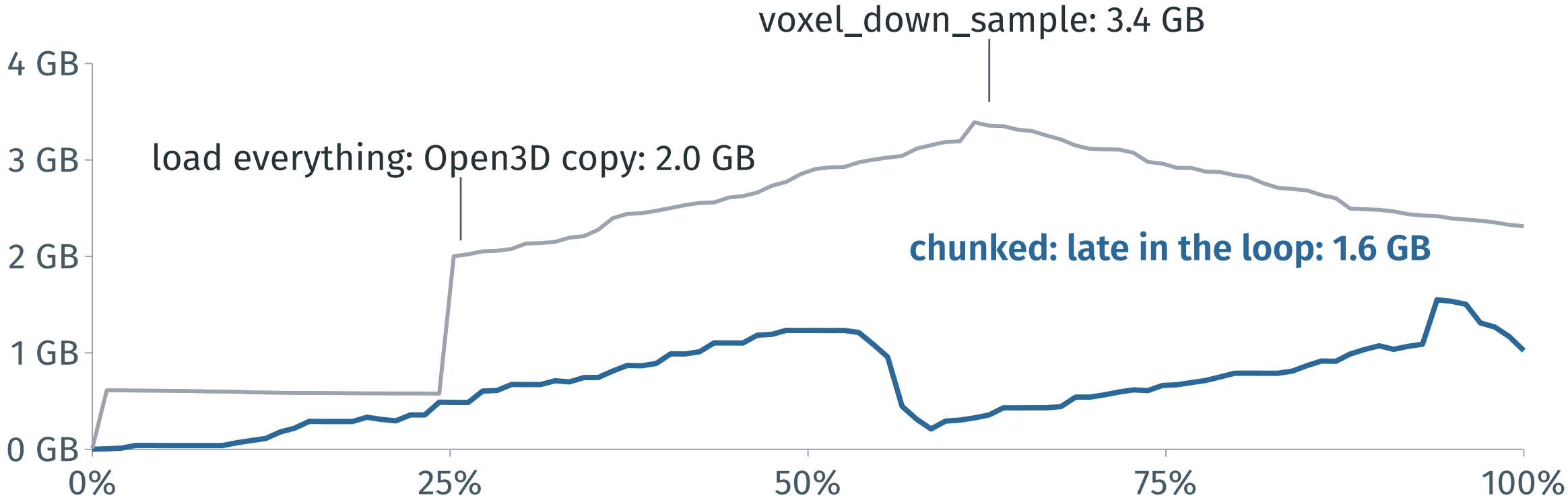
Stream chunks, keep the first point per voxel

```
v, keys, pts = 0.5, [], []                          # voxel size: 0.5 m
with laspy.open(path) as f:
    lo, hi = f.header.mins, f.header.maxs           # local origin, extent
    dims = ((hi - lo) // v).astype(np.int64) + 1    # voxels per axis
    for c in f.chunk_iterator(1_000_000):           # a million points
        xyz = np.c_[c.x, c.y, c.z] - lo
        ijk = (xyz // v).astype(np.int64).T         # voxel index per axis
        key = np.ravel_multi_index(ijk, dims)       # one int64 per voxel
        k, first = np.unique(key, return_index=True)
        keys.append(k); pts.append(xyz[first])      # first per voxel
k, first = np.unique(np.concatenate(keys), return_index=True)
points = np.concatenate(pts)[first]                 # earliest chunk wins
```

3.0 GB → 0.8 GB peak memory, 1 km AHN file

Conditions: Ryzen 7 5700X, Python 3.13.15, NumPy 2.5.3; 21 013 926 points, median of 3 runs, each in a fresh process; AHN5 1 km file (CC BY 4.0). 1 GB = 2^{30} B. Full: results/memory.json.

Why it works: memory over time



Profiled with Scalene. Load everything: memory follows the file. Chunked: memory follows what you keep.

Conditions: Ryzen 7 5700X, Python 3.13.15, Scalene 2.3.0; 21 013 926 points, median of 3 runs; AHN5 1 km file (CC BY 4.0). Scalene counts allocations, not resident memory (slide 26). Full: results/memory_scalene.json.

Six gotchas I hit building this

`read_point_cloud("x.laz")`	Empty cloud, no error
`pip install laspy`	LAZ files will not open
`xyz.astype(np.float32)`	1.6 cm steps in RD
`Vector3dVector(float32)`	39× slower copy
`segment_plane(seed)`	Differs unless probability=1
`classification == 1`	Class 1 is not trees

Take the four rules home

1 **Think in arrays, not points**

722 961 × 3, as views

slides 6, 11

2 **Look before you compute**

61 poles on the map, 3 found

slides 19, 30

3 **Make it small first**

3.0 GB → 0.8 GB peak

slides 13, 28, 29

4 **Let compiled code loop**

130× faster than a loop

slides 21 to 25

Copy the pipeline

Your loop is still running. Your coffee is cold.



github.com/Sloppy1999/pycon-nl-2026-3d-data-science

Sergio Peral
sergioperalgarijo99@gmail.com
github.com/Sloppy1999

Point cloud: AHN5, Actueel Hoogtebestand Nederland (ahn.nl), Het Waterschapshuis, CC BY 4.0. AHN file 135000_455000, cropped and processed.
Licence: creativecommons.org/licenses/by/4.0
[Optional: Title shared with F. Poux, 3D Data Science with Python, O'Reilly 2025; this talk is independent.]